SciDK: Architecture & Vision

A Reproducible Scientific Workflow Platform

The Core Idea

SciDK is not a database, a dashboard, or a file browser. It is a **reproducible scientific workflow platform** — a system that captures not just what data exists, but how it was understood, connected, and analyzed. Every piece of knowledge in SciDK has a traceable origin. The entire configuration of a project can be serialized and rehydrated. An agent reading that configuration can reconstruct not just the data, but the reasoning behind it.

This is a living, executable methods section.

The Fundamental Symmetry

The most important architectural insight in SciDK is the symmetry between **authoring** and **transparency**. Every type of intelligence in the system has two surfaces: a place where it is written, and a place where it is explained.

Script Type	Authors In	Transparency Layer	Output to KG
Interpreters	Scripts page	Files sidebar	Entity metadata + labels
Links	Scripts page	Links page	Relationships
Analyses	Scripts page	Results page	Enriched annotations
Plugins	Scripts page	Plugins page	Reusable logic

Scripts is the **workbench** — where all intelligence is authored, tested, and validated. The other pages are **transparency layers** — where that intelligence is visible and auditable at the point of use. A scientist who never touches code can still understand exactly what is shaping their view of the data.

This symmetry is a first-class design principle. It must be preserved as the system grows.

Navigation & The Conceptual Gradient

```
-SciDK-> Results Chats Maps Labels Links Files Scripts Plugins
```

The navigation reads left to right as a gradient:

Page	Primary Question	Primary User
Results	What have we learned?	Everyone (returning)
Chats	What does the data say?	Everyone
Maps	How is everything connected?	Everyone
Labels	What kind of thing is this?	Data stewards
Links	How do things relate?	Data stewards
Files	What data do we have?	Everyone (new users)
Scripts	How do I extend the logic?	Developers
Plugins	What intelligence is running?	Everyone

Results is the landing page for returning users — it shows the current state of the project at a glance. **Files** is the natural entry point for new users discovering what data exists. **Plugins** loops back: technically the most complex page to build, but the most accessible to read. A scientist who never opens Scripts can meaningfully use Plugins to understand what the system is doing.

The Workflow: Data to Knowledge

```
Files (scan)
  ↓ Interpreters extract metadata + assign entity types
Labels (organize)
  ↓ Entity types confirmed, taxonomy defined
Links (connect)
  ↓ Relationships created between entities (wizard or script)
Analyses (enrich)
  ↓ Computations run, results pushed to KG with provenance
Results (communicate)
```

↓ Self-assembling transparency layer shows what was learned

Maps (visualize)

↓ The connected knowledge graph, explorable and queryable

Chats (interrogate)

↓ Natural language questions answered against the full graph

Each stage builds on the last. The KG is not a static import — it is itself an artifact of the analyses and connections that have enriched it over time.

The Scripts Page: One Workbench, Four Outputs

All intelligence in SciDK is authored in the Scripts page. Scripts are categorized by their contract and output:

Interpreters

Contract: `interpret(file_path: Path) -> dict`
Purpose: Take a file, return structured metadata and an entity type label.
Output: Entity nodes in the KG with rich metadata.
Transparency: Files sidebar — "Interpreted by: FASTQ Interpreter ↗"

Links

Contract: `create_links(source_nodes: list, target_nodes: list) -> list`
Purpose: Take two sets of nodes, return relationship triples.
Output: Edges in the KG connecting entities.
Transparency: Links page — unified view of wizard and script links with type badges.

Analyses

Contract: `run(context: AnalysisContext) -> None`
Purpose: Query the KG, compute insights, register visual panels, optionally write back.
Output: Enriched KG nodes + visual panels on the Results page.
Transparency: Results page — self-assembling panels with full provenance.

Plugins

Contract: `run(context: dict) -> SciDKData`
Purpose: Reusable logic modules called by other scripts.
Output: `SciDKData` objects (dict, list, DataFrame) consumed by callers.
Transparency: Plugins page — active/inactive/draft with dependency graph.

The Context Object: Clean Interface for Script Authors

Analysis scripts receive an `AnalysisContext` object that provides access to the platform without exposing infrastructure:

```python
def run(context):
    # Query the knowledge graph
    results = context.neo4j.query(
        "MATCH (f:File) RETURN f.extension as ext, count(*) as count"
    )

    # Register a visual panel on Results page (deferred until success)
    context.register_panel(
        panel_type='table',
        title='File Distribution by Extension',
        data=results,
        visualization='bar_chart'
    )

    # Write back to KG — provenance auto-injected
    context.neo4j.write_node(
        label='FileStats',
        properties={'total_types': len(results), 'analyzed_at': context.ran_at}
        # __source__, __script_id__, __execution_id__, __created_at__ added automatically
    )
```

Three things happen in ten lines: query, communicate, enrich. Panel registration is **deferred** — panels are only written when the script completes successfully. Partial failures leave no misleading trace on Results.

SciDKData: Universal Plugin Return Type

Plugins return a `SciDKData` object. Plugin authors don't need to know this — `load_plugin()` wraps their output automatically:

```python
```

```
# Casual scientist writes this — auto-wrapped
def run(context):
    return {'gene': 'BRCA1', 'count': 42}

# Experienced developer writes this — explicit
from scidk.core.data_types import SciDKData
def run(context):
    return SciDKData().from_dataframe(my_df)
```

`SciDKData` accepts dict, list, or DataFrame as input and exposes `.to_dict()`, `.to_list()`, `.to_dataframe()`, `.to_json()` regardless of input type. The interface is consistent; the input format is irrelevant.

Provenance: Every Node Knows Its Origin

All KG writes through the analysis context automatically carry provenance metadata:

```
__source__:       'analysis'
__script_id__:    'file_distribution_v2'
__execution_id__: 'exec_abc123'
__created_at__:   1708531200.0
__created_via__:  'scidk_analysis'
```

The same pattern applies to Interpreter writes (source: 'interpreter') and Link writes (source: 'link'). This means every node and relationship in the graph carries a traceable origin. Six months later, anyone can answer: "where did this come from?"

The Results Page: Self-Assembling Transparency

Results is not a dashboard that someone designs. It is a page that assembles itself from what has been done.

Schema Summary (top): Real-time query of the KG — entity types and counts, relationship types and counts, last updated timestamp. The state of the project at a glance.

Analysis Panels (below): Each panel corresponds to a completed analysis. It shows the script name, when it ran, and a rendering of what it produced (table, metric, figure). Panels are ordered chronologically. No manual curation.

Empty State: For new projects, a clear message: "No analyses have been run yet. Go to Scripts to create your first analysis." This is an explicit, visible state — not invisible emptiness.

Panel Removal: A cleanup modal shows what the analysis found before asking for deletion. Scientists should understand what they're removing.

In science, showing your work is the result. The Results page makes this literal.

The Plugins Page: Platform-Wide Intelligence Visibility

Plugins are organized in three sections:

Active — validated, activated, currently shaping system behavior. Shows name, description, validation timestamp, and crucially: "Used by: FASTQ Interpreter, 3 Link scripts." The dependency graph is visible.

Available but Inactive — validated but not switched on. Shows a link to activate.

Draft / Failed — in progress or broken. Links to Scripts page for editing. Not surfaced in production use.

The dependency data comes from a SQL junction table (`script_dependencies`) populated by AST scanning at validation time. When a script calls `load_plugin('normalizer')`, that dependency is recorded. When the script is edited and reset to draft, the dependency row is cleared. The "Used by" display is always accurate relative to validated scripts only.

The Links Page: Two Complementary Approaches

Links has a unified view combining two fundamentally different creation modes:

Wizard Links — declarative, visual, form-based. Created in the Links page wizard. Best for non-developers and straightforward property matching.

Script Links — imperative, code-based. Created in Scripts page (category: links). Best for complex logic, similarity algorithms, inference, external APIs.

Both appear in the same list with type badges (WIZARD / SCRIPT) and validation status. Clicking a script link shows a read-only detail view with redirect to Scripts for editing. Execution from the Links page uses Cypher injection protection on relationship type strings.

The message: two tools for different jobs, working together seamlessly.

The Files Page: Interpretation at the Point of Discovery

When a file is selected in the Files browser, the right sidebar shows metadata plus — if an interpreter has run — attribution:

```
📄 sample_R1.fastq
─────────────────────────
Entity Type:    SequencingFile
Interpreted by: FASTQ Interpreter ↗
─────────────────────────
read_count:     1,847,293
format_version: 1.8
paired_end:     true
─────────────────────────
🔬 Interpreters          [▼]
```

For unrecognized files: "⚠️ No interpreter assigned — click to explore available interpreters."

The `[▼]` accordion opens an interpreter modal — a non-destructive overlay where the user can select interpreters, run a preview (no KG writes), review the extracted metadata, and then commit to the graph. Preview and commit are decoupled endpoints. Interpretation is always safe to explore.

Chat: Unified Interface for Data and System State

Chat can answer questions about research data ("find all FASTQ files from 2023") and — through the `/api/system/` endpoints — questions about system state:

- "Which interpreter handles my .fastq files?" → `GET /api/system/file-interpreter/sample.fastq`
- "What would break if I deactivate genomics_normalizer?" → `GET /api/system/plugin-dependencies/genomics_normalizer`
- "Show me all active plugins" → `GET /api/scripts/active?category=plugins`

The principle: **everything safe to access should be accessible to Chat**. Build the human UI first, then expose the same data through tool-callable endpoints. The architecture for this is already implicit in the API structure — it just needs to be applied consistently.

Data Architecture: SQL for Infrastructure, KG for Data

Data Type	Storage	Rationale
Research entities, relationships	Neo4j KG	Graph traversal, semantic queries
Script metadata, validation status	SQLite	Fast, relational, infrastructure
Plugin dependencies	SQLite (`script_dependencies`)	Small, relational, never user-facing
Analysis panel registry	SQLite (`analysis_panels`)	Fast lookup, linked to execution history
File scan results, basic metadata	SQLite	Pre-KG indexing layer

The Knowledge Graph holds research data. SQLite holds infrastructure metadata. These concerns never cross.

The Validation Lifecycle

All scripts follow the same lifecycle before they can affect production:

```
Draft → [Edit resets to Draft]
  ↓ Validate (AST import check + contract tests in sandbox)
Validated
  ↓ Activate
Active → appears in transparency layers, callable by other scripts
```

Only validated scripts appear in Settings dropdowns, the Plugins page active section, and the Files interpreter modal. Only active scripts are callable via `load_plugin()`. Editing a validated or active script automatically resets it to Draft and deactivates it.

The sandbox is pragmatic for MVP: subprocess execution with a 10-second timeout and an AST-validated import whitelist. Full container isolation is post-MVP.

The Reproducibility Artifact (Post-MVP)

The complete SciDK project configuration — interpreters assigned, links defined, analyses run, KG schema — can be serialized to a human-readable JSON or YAML file. This file is the executable methods section. Importing it into a fresh SciDK instance rehydrates the project: re-scans files, re-runs analyses,

rebuilds the graph. An agent reading the configuration file can reconstruct not just the data but the reasoning that produced it.

This is the long-horizon vision. The Results page is its immediate precursor — it makes the methods section visible before making it portable.

Design Principles

These principles govern all architectural decisions. When a future choice is unclear, consult this list.

1. **SQL for Infrastructure, KG for Data** — Dependencies, script metadata, validation status live in SQLite. Research data, entities, relationships live in Neo4j. These concerns never cross.
2. **Chat-Accessible by Design** — Every tool built for users has a clean query endpoint Chat can call. Build the human UI first, then expose the same data as a tool-callable API. Everything safe to access should be accessible.
3. **Decoupled Preview + Commit** — Non-destructive exploration before graph writes. Run interpreters, preview link creation, see analysis results — none of it commits until explicitly confirmed.
4. **Explicit Degraded States** — "No interpreter assigned," "no analyses run," "no results yet" are visible states with clear calls to action. Never show nothing.
5. **Transparency Without Clutter** — Intelligence is visible at the point of use. Interpreters explained in the Files sidebar. Links explained on the Links page. Analyses explained on the Results page. Plugins explained on the Plugins page. Not buried in Settings.
6. **Validation Guarantees Visibility** — Only validated scripts appear in transparency layers. Draft and failed scripts are visible to developers in Scripts and Plugins but do not surface in production use.
7. **Every piece of knowledge has a traceable origin** — File (Interpreter), rule (Link), computation (Analysis), human assertion (Labels). Provenance is a first-class property of all KG nodes and relationships. Double-underscore naming convention: `__source__`, `__script_id__`, `__created_at__`.
8. **The configuration is the methods section** — The complete set of scripts, rules, and analyses that produced the current KG state can be serialized, shared, and rehydrated. A future agent can read it and reconstruct the reasoning.
9. **Authoring and transparency are symmetric** — Every script type has exactly one authoring surface (Scripts) and exactly one transparency layer (the relevant page). This symmetry must be preserved as new script types are added.

10. **The page assembles itself** — The Results page, the Links page unified view, the Plugins dependency display — none of these are manually curated. They are derived automatically from execution history and validation state. The system tells its own story.

The Story the System Tells

When a scientist opens SciDK on a mature project:

1. **Results** — "Here is what we have learned. These analyses ran. This is the state of the knowledge graph."
2. **Chats** — "Ask me anything about the data."
3. **Maps** — "Here is how everything connects."
4. **Labels** — "These are the entity types we have defined."
5. **Links** — "These are the rules and algorithms that create relationships."
6. **Files** — "These are the raw files. Each one was interpreted by a specific script."
7. **Scripts** — "This is how the intelligence was written and validated."
8. **Plugins** — "This is what is currently running under the hood."

Every page answers a different question. Together they answer the only question that matters: **how do we know what we know, and can we trust it?**

SciDK — Science Data Kit
Architecture document reflecting MVP design decisions as of February 2026.